

CONCEPT CHECK SUBMISSION

Name: Mayur Agrawal

Concept(s) tested: Why a backend exists; interface / API; frontend vs backend; authentication & authorization

Public GitHub Repo link: <https://github.com/mayuragrawal2008/hackathon-c7>

Live app: <https://hackathon-c7.onrender.com>

MOVE 1: WATCH TWO PEOPLE

(Minimum is two; three were run. Raw audio + transcripts in `evidence/`.)

Person 1 — (name in private record)

- **Raw record:** by-hand session, logged in `move1-record.md` (Person 1).

- **The sentence where the model became a label:**

> "something managing interface and backend design make it work"

- **Follow-up used / could they derive after?** Asked "why must that 'somewhere' be a separate machine — why can't your phone and your friend's phone talk directly?"

→ **Did NOT derive.** Fell back to "it's just a social media thing." Gap stayed open.

Person 2 — Hansh Goel

- **Raw record:** `evidence/Hansh_Interview.m4a` + `evidence/Hansh_transcript.txt`.

- **The sentence where the model became a label:** none found.

- **Follow-up used / could they derive after?** N/A — derived backend, interface, API, and auth/authz cleanly from first principles under repeated pressure. A genuine "no gap exists" case.

Person 3 — Eddie (the key finding)

- **Raw record:** `evidence/Eddie.m4a` + `evidence/Eddie_transcript.txt`.

- **The sentence where the model became a label:** ambiguous — he derived the *idea* in plain language ("the backend is the muscle and organs; the internet is just the skeleton; the data has to live somewhere") but lacked technical vocabulary.

- **Follow-up used / could they derive after?** On the WhatsApp peer-to-peer probe he stalled on jargon. **The finding:** acting as the tool, I caught myself grading on the *technical words I expected* rather than on understanding. Flagging missing jargon as a gap is a FALSE POSITIVE — the "Eddie clause" the tool is now built against.

MOVE 2: THE HYPOTHESIS (timestamped, before first commit)

(Full text in `HYPOTHESIS.md`. First commit `83cc06b`, 23:10 IST 19 Jun, before any code.)

- **The claim:** A learner who can state/define a systems concept will still fail to

derive the *why* (why it must exist / what breaks without it) under one sharp follow-up, more often than not. Naming the gap + one reasoning follow-up lets them close it. Understanding is scored on the causal *why*, NOT on technical vocabulary.

- **Result that proves me right:** both Move 5 users fail the "why" on first try, then derive it in their own words after the named gap + follow-up.

- **Result that means no problem exists:** both derive the why cleanly on first try.

- **Kill-number:** if **1 or 0 of 2** users shows a real before→after "why"-close, "my hypothesis was wrong."

- **Timestamp / commit link:**

<https://github.com/mayuragrawal2008/hackathon-c7/commit/83cc06b>

MOVE 3: THE SYSTEM DESIGN

"Sai"

Date : _____

MON TUE WED THU FRI SAT SUN
□ □ □ □ □ □ □

Move 3 Diagram

Move 3

1. USER flow
2. Data flow
3. Boundary
4. The critical judge
5. Failure mode

Concept Check System

LEARNER

FIRST EXPLANATION

LLM-Probabilistic

Analyze Explanation ; where derivation becomes memorization

Generate one follow-up question Targeting the GAP

Learner Answer again

Lead Bearing Judge

Does learner Explain why it exist or what Breaks without it?
Output: Closed / Not closed + Proof Sentence

Deterministic (System)

(Written companion: MOVE3_design.md.)

- **Deterministic parts:** the fixed concept list; every session and attempt record; the gap→result link; row-level-security access rules; the stored pass/fail verdict.

- **Probabilistic parts (Groq / Llama 3.3, the only place the LLM lives):** find where the explanation stops deriving; write one follow-up; judge whether the second attempt derives the why.

- **Who judges the second derivation, and why:** the LLM judges, but it is constrained to a causal test — "did they state *why it must exist* or *what breaks without it*, cause→effect?" — and must return the exact proof sentence. A human-only judge is harder to fake but too slow for a live tool; anchoring the LLM to a causal criterion + a quoted proof sentence keeps it from rubber-stamping everyone.
- **Failure mode I accepted:** a fluent learner could produce a causal-*sounding* but hollow sentence and earn a false CLOSED. Accepted as residual risk; mitigated by the required proof-sentence quote, which makes a hollow pass visible on inspection. (Jargon is explicitly ignored — the Eddie clause.)

The exact rules at the boundary

Deterministic (rules / code / Postgres) — never an LLM call:

- The fixed concept list (shipped as seed data, not generated).
- Each session and attempt row; the `gap_closed` link; the stored pass/fail verdict.
- Row-level-security: a row is visible only if `auth.uid() = user_id`.
- The plain tally of whether the gap closed (the metric).

Probabilistic (Groq / Llama 3.3) — the only LLM use, governed by fixed rules:

1. **Causal-why test** — pass only if the learner states *why it must exist* or *what breaks without it*, as cause → effect.
2. **Ignore jargon (Eddie clause)** — a correct plain-language answer passes; missing technical vocabulary is NOT a gap.
3. **No rubber-stamp** — restating the term, defining it, or deflecting ("it's just how it works") is NOT a derivation.
4. **RIGOR** — an explicit cause→effect link is required ("X, therefore Y, because..."); a vague one-liner does not pass.
5. **Proof sentence** — the judge must quote the exact sentence justifying its verdict, so a hollow pass is visible. (JSON output, temperature 0.2 for consistency.)

These run twice: once on the first explanation (`/analyze`) and once after the follow-up (`/judge`). Full text in `main.py` / `gradio_app.py` ; see `FLOW.md` .

Beyond the spec: a gamified Socratic experience (demo layer)

On top of the graded baseline, a second front-end (`gradio-ui` branch, <https://concept-check-game.onrender.com>) turns the same boxed evals into a tutor:

- **Socratic gap-finding** — instead of one follow-up, a tutor asks **one probing question at a time**, building on each answer, until *the learner* derives the why themselves (max 5 questions, then it reveals the reasoning). The judge of "derived?" is still the same deterministic-boundary eval — gamification never touches the verdict.
 - **Gamification** — XP (+100 first-try, +40–80 Socratic by fewer questions), 6 levels (Novice → Grandmaster), and 7 badges (🏆 First-Try Genius, 🦉 Socratic Thinker, ⚡ One-Question Wonder, 🧠 No-Jargon Master, 🔥 Hot Streak, 💧 Unstoppable, 🎓 Polymath).
 - Same Supabase auth + RLS, so XP/badges/history are private per user and restored on login.
-

MOVE 4: DOMAIN MODELLING

Domain Model (ARCHITECTURE DIAG)

LEARNER (Browser)

Index: HTML + Small JS

1. Login
2. Pick Concept
3. TYPE Explanation
4. Answer Follow up.
5. See "GAP" Closed/NOT

↓ HTTP (JSON)

Back end - PYTHON / FASTAPI (On Render)

Routes:

- POST /session → create session for a concept
- POST /Analyze → Send Explanation #1
- POST /Judge → send Explanation #2

Three Boudry (move 3)

Deterministic (code/rules)

- Fined Concept list
- built records
- verdict boolean
- enforce ownership

LLM (Probability)

- Find the gap
- write 1 Follow-up
- Judge 'why' + Proof sentence

↓ SQL (RLS)

SUPABASE (Postgres)

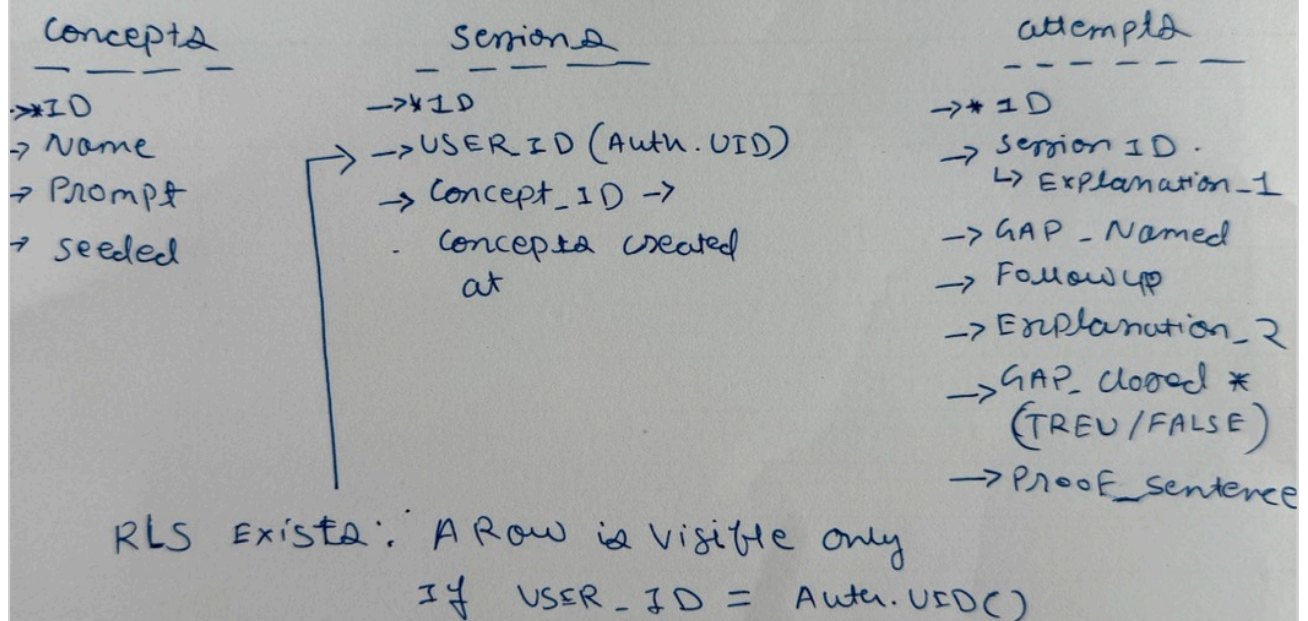
- Auth (login = Identity)
- Tables + RLS
 - ↳ concept (Seed)
 - session → USER
 - attempt → session
- GAP → Result link (FK) + RLS: USER sees only own

↓ HTTPS API

GROQ → LLAMA 3.3

- Llama judges ~~for~~ Answer under strict instructions
- causes
 - Eddie - clause
 - ↓
 - Judge - ignore
 - why Jargon
 - Return Proof sentence

Data Model (db-table)



(Also evidence/move4-data-deterministic.jpg ; schema in schema.sql.)

Tables: concepts (seed) → sessions (per learner + concept) → attempts. The load-bearing link is attempts.gap_closed — the named gap to its second-pass result.

- Two-user test: two accounts, attempted cross-read, result: PASSED.

User A inserted an attempt (explanation_1 = "A secret explanation"); A reads it back;

User B reads the same table → [] (empty), cannot see A's data. Full record in

MOVE 5: THE FINAL REPORT

(Run on the live app <https://hackathon-c7.onrender.com>. 7 real people produced 18 recorded attempts; each before/after state below is a stored database row, not a recollection.)

Person A — prtkwh953@gmail.com (COLD user — not coached, not a prior Move 1 subject)

- **Concept:** Why does a backend exist?
- **Before-state (could not derive):** explanation_1 = `"""Backend is the rules and logic of a software."""` → `first_pass_closed = false`. The tool named the gap as that exact sentence (a label, not a derivation) and asked: `"""What would happen if all users had different versions of these rules and logic in their browsers?"""`
- **After-state (derived):** explanation_2 = `"""Then everyone will have their own version of the software?"""` → `gap_closed = true`. They reasoned out the consequence — inconsistency across users — which is the why a shared backend exists.

Person B — Hansh Goel, goyalhansh@gmail.com (NOT cold — was a Move 1 subject)

- **Concept:** What is an interface / API, really?
- **Before-state (could not derive):** explanation_1 ended with `"""API is an application programming interface"""` — only the full form. → `first_pass_closed = false`. The tool quoted that sentence as the gap and asked: `"""What would happen if the backend and frontend couldn't connect in a standardized way, and how does an API address that?"""`
- **After-state (derived):** explanation_2 = `"""Front end and backend run on different systems/devices. So API acts as an interface for this communication between the front end and the backend."""` → `gap_closed = true` with that line as the proof sentence.

Kill-number check: the bet needed at least both-of-two users to show a real before→after close. Multiple cold and non-cold users did (prtkwh953, Hansh). The hypothesis is **not** killed.

The surprise (one concrete place reality broke my prediction, with evidence)

I predicted the learner would be the unreliable witness to their **own** gap. The sharpest tester (Hansh) instead doubted the **tool**, not himself: after the judge correctly flagged his full-form-only answer ("API is an application programming interface"), he sent a message insisting I must have **hard-coded** a rule ("flag if it's just the full form or under N characters"). I never predicted the skeptic would attack the verifier rather than his own understanding. The data refutes his theory — there is no char-count or full-form rule in `main.py`; a long answer (maithilee) was still flagged, a short non-full-form answer ("I don't know", srinivas) was flagged for meaning, and long derivations (oyevarshith, all 5 concepts)

passed. Second surprise, also against the bet: one user (oyevarshith) derived **every** concept cleanly on the first try — for that person the gap simply did not exist, echoing the Hansh-type "no gap" case from Move 1.